



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 20-21: Sensor Fusion and Kalman Filters

Marius Juston

Monday, April 6th, 2026

Talk to the person next to you and answer these questions.

- What assumptions were made during path planning / trajectory planning?
- What sensors on the robot help determine it's the location in the world?
- When could the odometry and the IMU give different values? Which one should you trust?
- If your IMU says you accelerated forward, but LiDAR scan matching says you didn't move, what should the system believe?

Office Hours:

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

Lab: Finishing Lab 6 this week. This a 3-week lab!

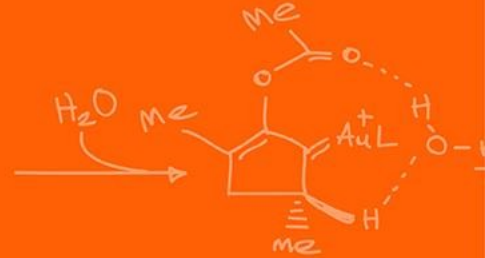
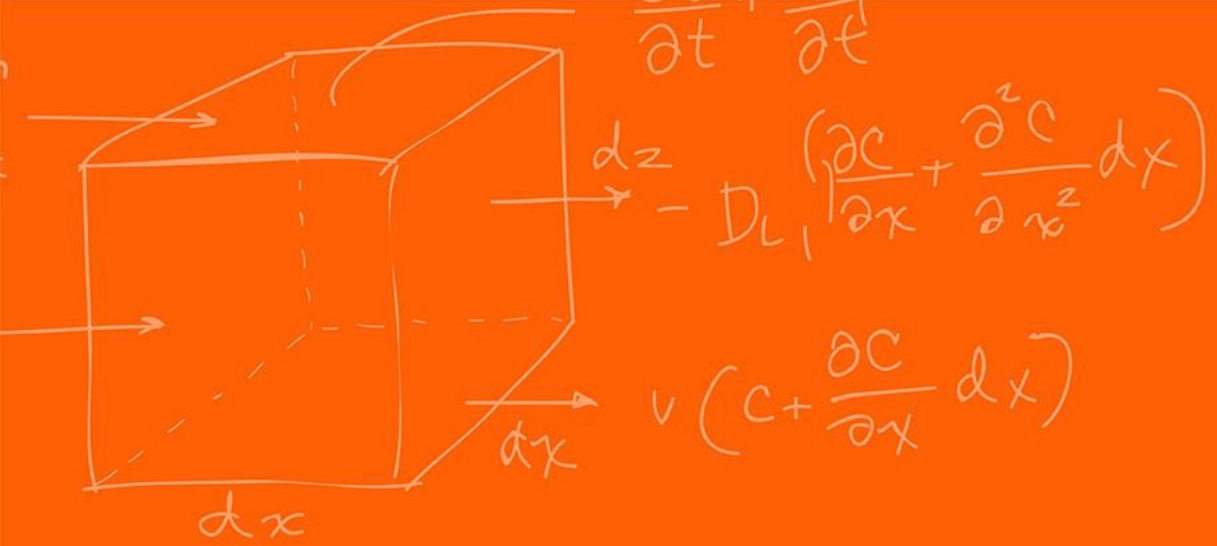
- Don't forget to have the Lab LabVIEW application implemented before lab.

Homeworks:

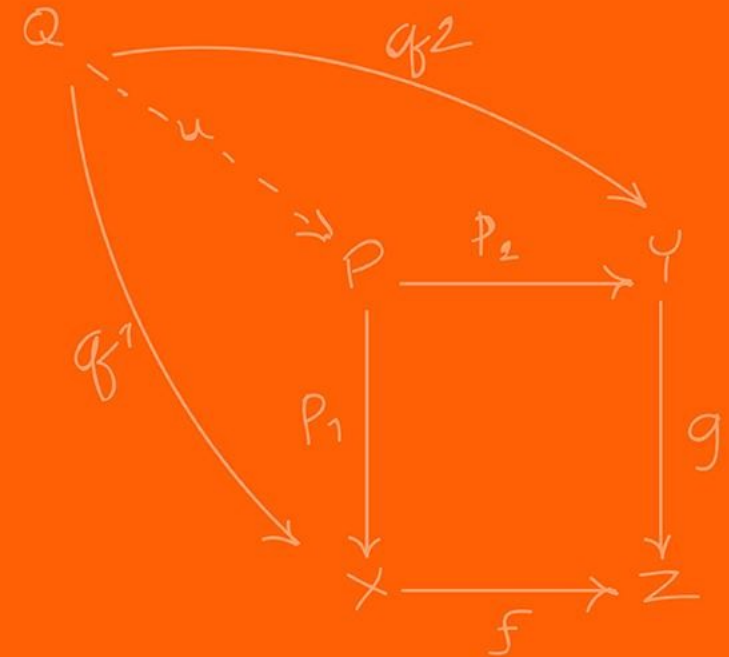
- Check-offs for [homework 4](#) Ex 4 are due **April 7, 5PM (The end of office hours)**
- Remainder of [homework 4](#) due on Gradescope on **April 8, 9AM**
- Next homework is primarily the **personal project** due on **April 22, 9AM**.
 - I would highly recommend you thinking now and maybe even starting it now if you have a good idea already!

Questions?

Homework, Lab, LabVIEW?



Sensor Fusion



In the real world, motors don't spin exactly as commanded due to friction and voltage drops.

Sensors have electrical noise, mechanical imperfections, and environmental interference.

What can cause the LiDAR to fail?

- Reflective / absorptive surfaces: glass, mirrors, black materials
- Fog, rain, dust, snow
- Fast motion relative to scan frequency

What can cause the odometry to fail?

- Slippery terrain for wheel slip

When can cause the IMU fail?

- IMU drift
- Saturating the sensors

If we blindly execute our path plan open-loop, the robot will quickly crash.

We need **Localization**: determining the robot's state (position, velocity, orientation) in the real world.

To solve the localization problem, SE 423 robots use multiple sensors.

Proprioceptive sensors measure internal states: Wheel Encoders (Odometry) and IMUs.

Exteroceptive sensors measure the external environment: LiDAR.

No single sensor is perfect; they all lie to us in different ways.

Wheel encoders count the rotations of the robot's wheels. By knowing wheel radius and track width, we calculate forward kinematics to estimate position. It provides **high-frequency**, smooth data for short-term movement.

Similarly, **IMUs** generate **high-frequency**, smooth data for short-term movement based on the integration of acceleration.

LiDAR, unlike odometry and IMUs, LiDAR gives us **absolute** measurements relative to the environment.; however, due to the additional data, the sensor runs at a lower frequency (e.g., 10Hz) compared to an IMU (e.g., 500Hz).

If we are in a perfectly square, featureless hallway, can LiDAR tell us our position along the hallway? What about the IMU / odometry?

We have **fast, smooth** sensors that drift (IMU, Odometry).

We have **slow**, computationally heavy sensors that don't drift (LiDAR).

How do we combine them?

Simple averaging doesn't work because their reliability changes over time.

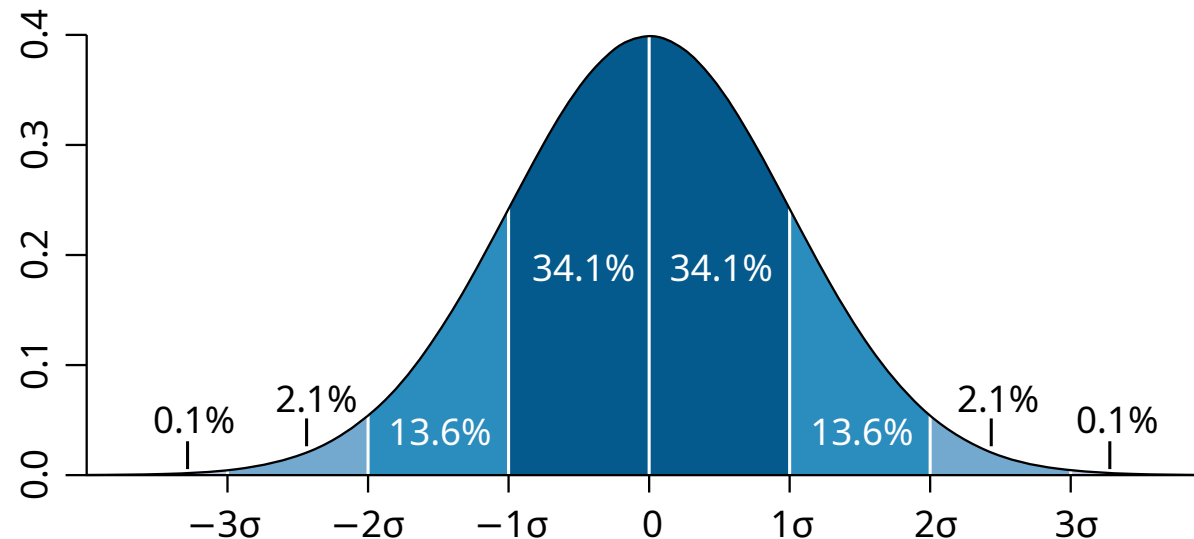
If 2 sensors given measurements give different values which, should you trust?

What if your sensors give you the confidence of their measurements? I.e. the LiDAR is 75% sure of its position and the IMU / Odometry are, in total, 25% sure of their positions?

Instead of saying robot is exactly at “ $X = 5$ ”, we want to say “We are 90% sure that the robot is near $X=5$ ”.

We represent our state as a probability distribution.

We use the Gaussian (Normal) Distribution, defined by a mean (μ) and variance (σ^2).



https://en.wikipedia.org/wiki/Normal_distribution

Why Gaussians?

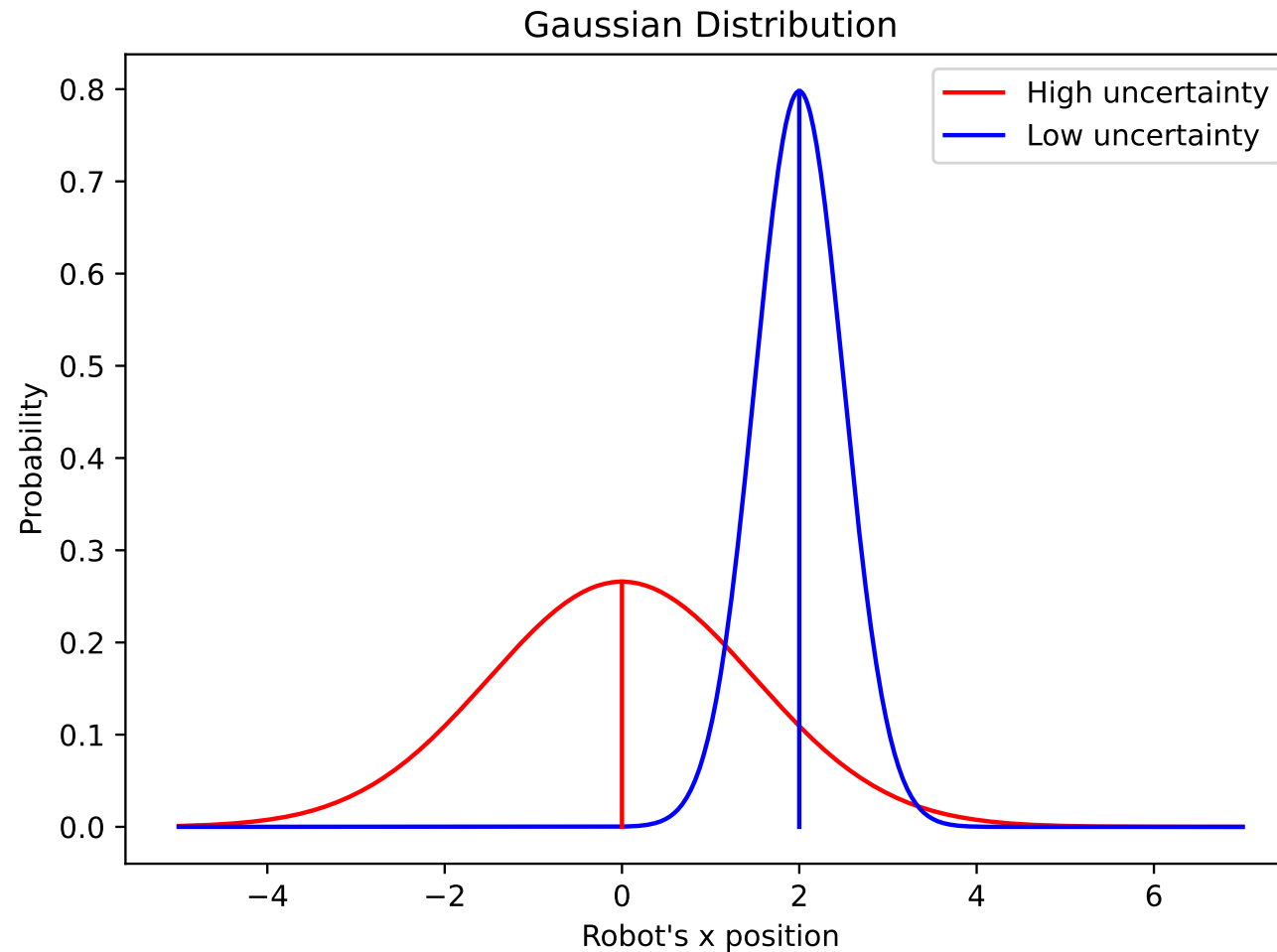
The **mean** (μ) represents our best guess of the robot's state.

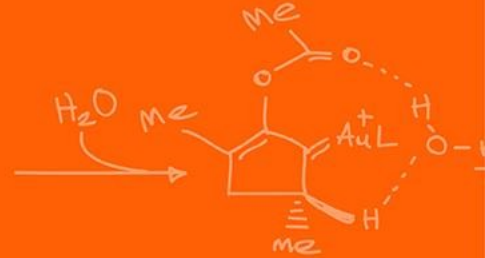
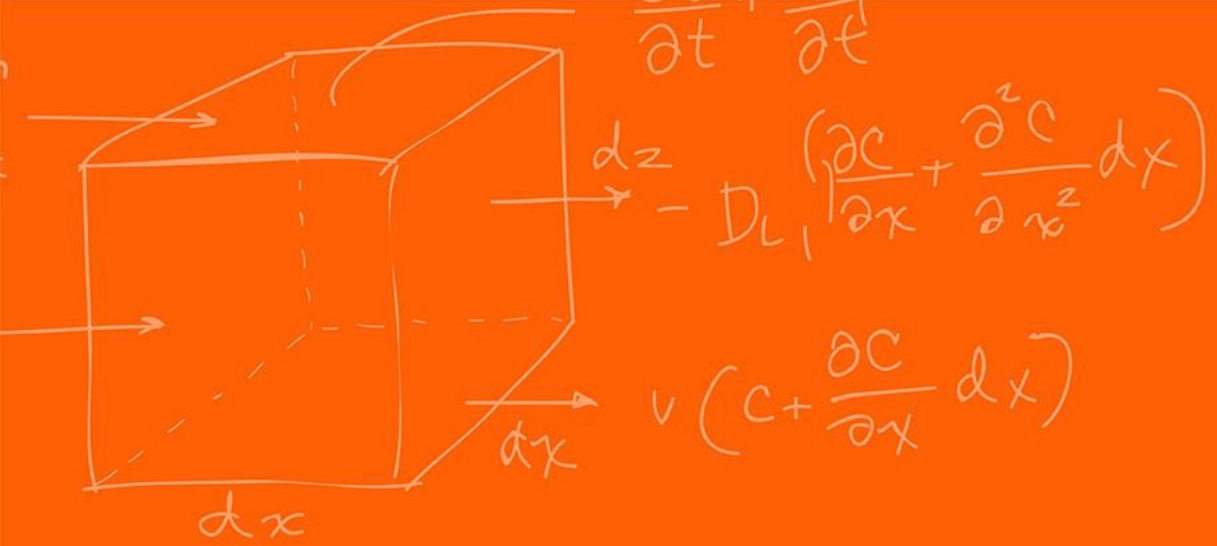
The **variance** (σ^2) represents our uncertainty; a wide curve means we are lost, a narrow curve means we are confident.

Gaussians are mathematically convenient: multiplying two Gaussians results in a new Gaussian!

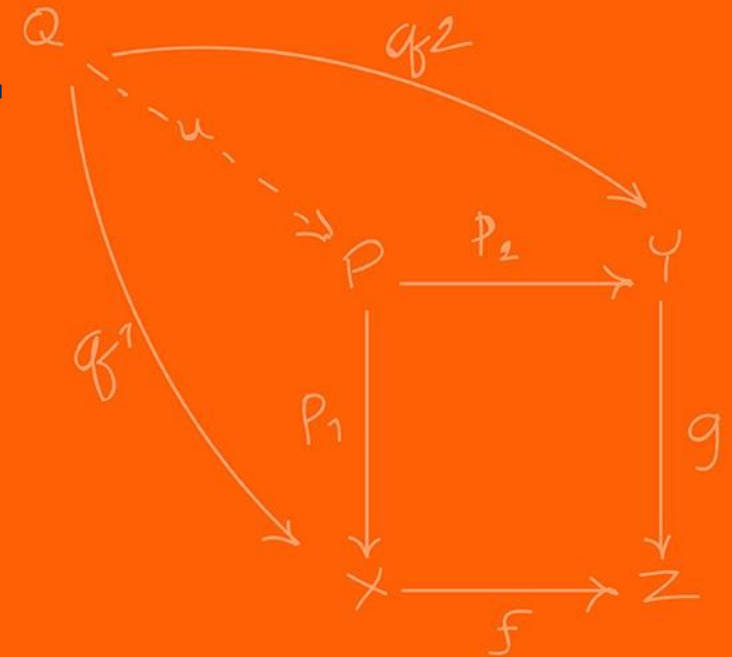
In addition, a lot of math can be derived in closed form when using Gaussians (most other more complex probability distributions do not allow for this)

Which sensor would you trust more?





1D Kalman Filter

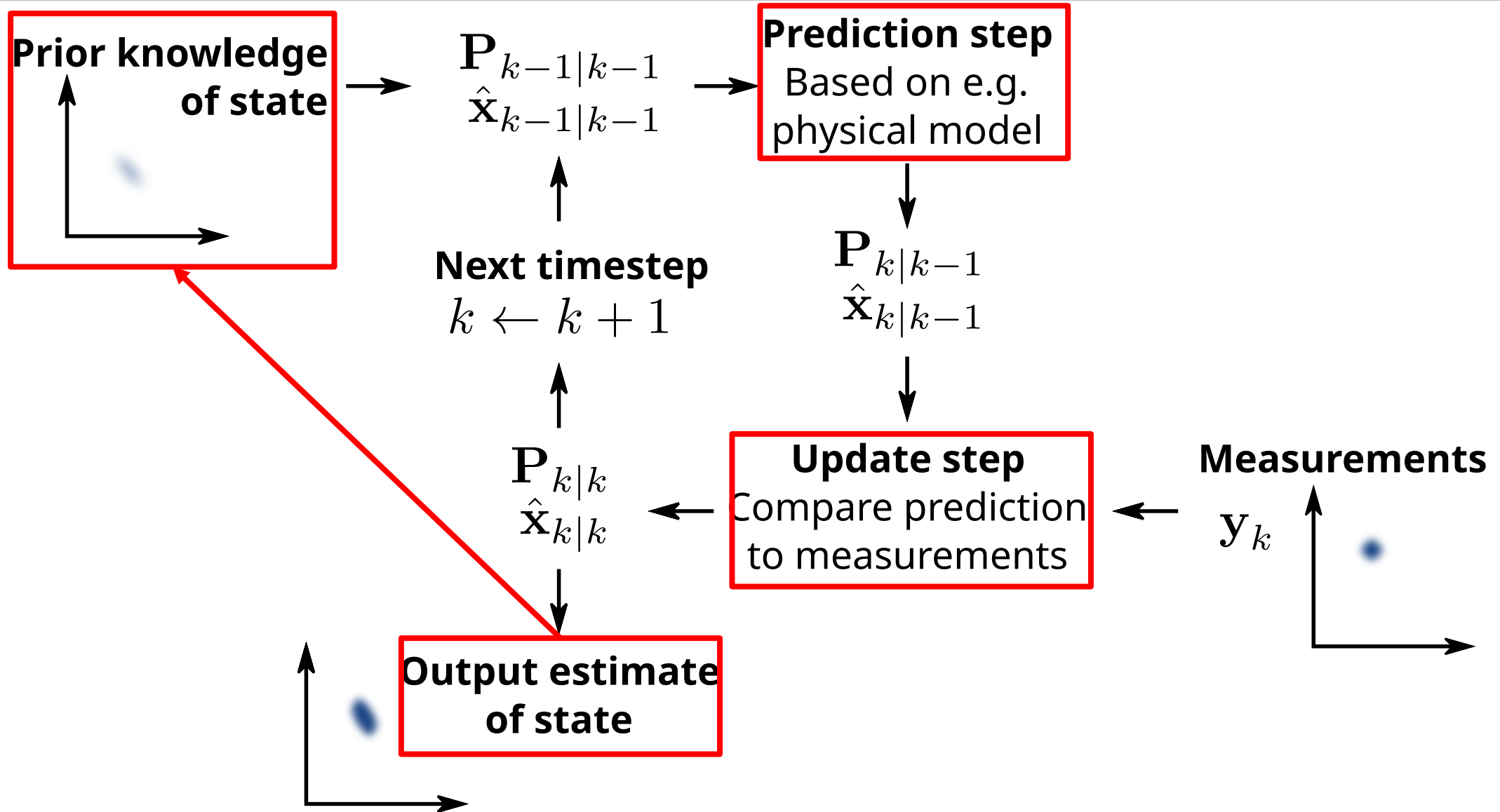


The Kalman Filter is an optimal estimator for **linear** systems subject to **Gaussian** noise.

When would the Kalman Filter not be appropriate?

If your sensor or system is non-linear and / or does not follow a Gaussian noise distribution! Always understand how your sensors / system acts and the underlying assumptions for algorithms!

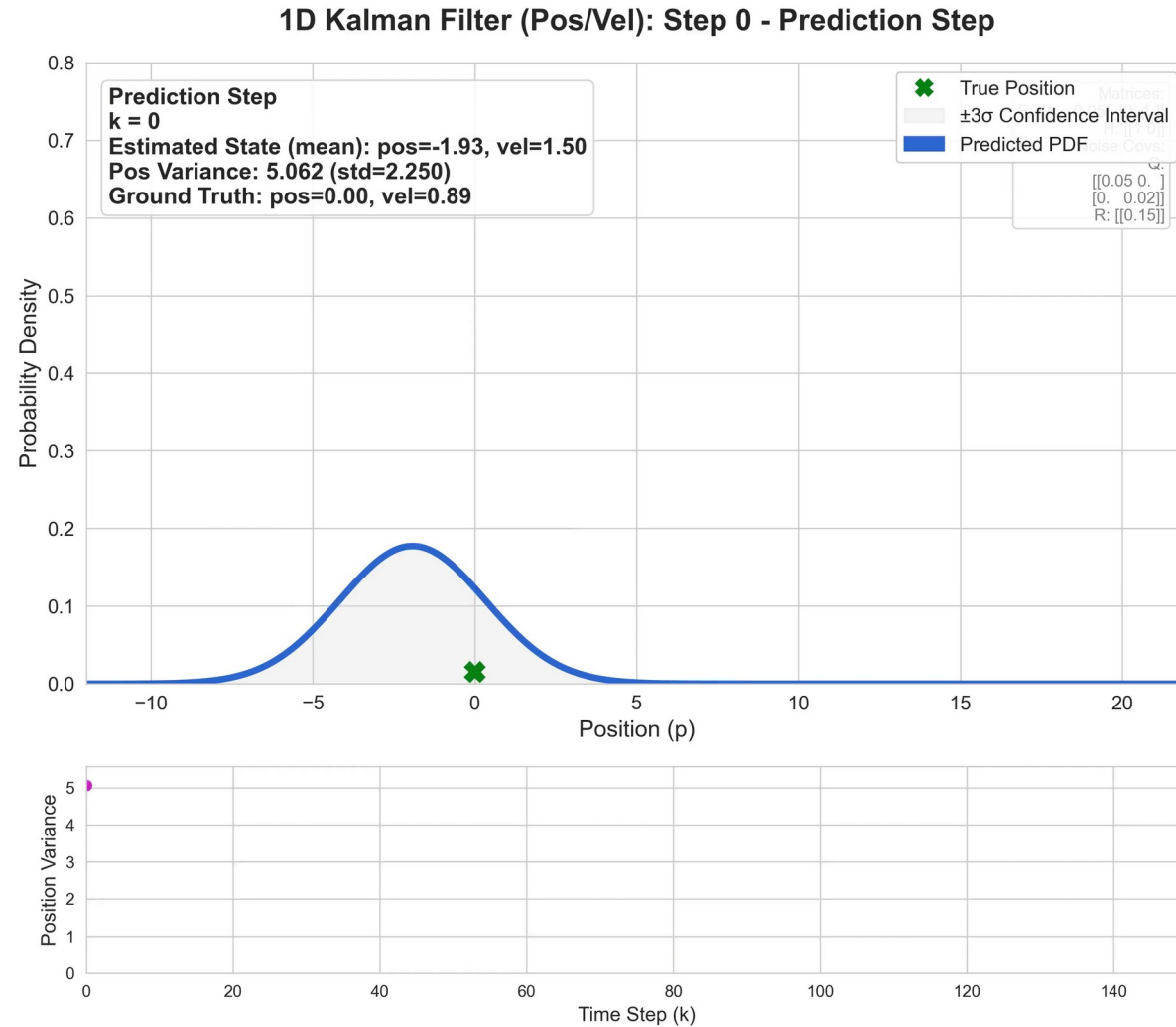
It operates in a continuous, two-step loop: **Predict** and **Update**.



Step 1: The robot **moves**. We use our fast, noisy sensors (Odometry/IMU) to **Predict** where we are. Our uncertainty grows.

Step 2: The robot **looks**. We take a measurement with our absolute sensor (LiDAR) to **Update** our belief. Our uncertainty shrinks.

Rinse and repeat thousands of times a second.



Predict

Last lectures, we covered algorithms like A*, Dijkstras, Hybrid A* for path planning.

We learned to compute an optimal, collision-free trajectory from Point A to Point B.

However, path planning operates in a perfect, simulated world!

The **fundamental assumption** we made about the robot to execute these plans were:

Perfect localization and actuation!

We use a physical model of the robot (kinematics) and our control inputs (u) to guess the new state.

Because we know our model and our encoders are imperfect, we actively increase our uncertainty covariance (P).

This represents "flying blind" between reference measurements.

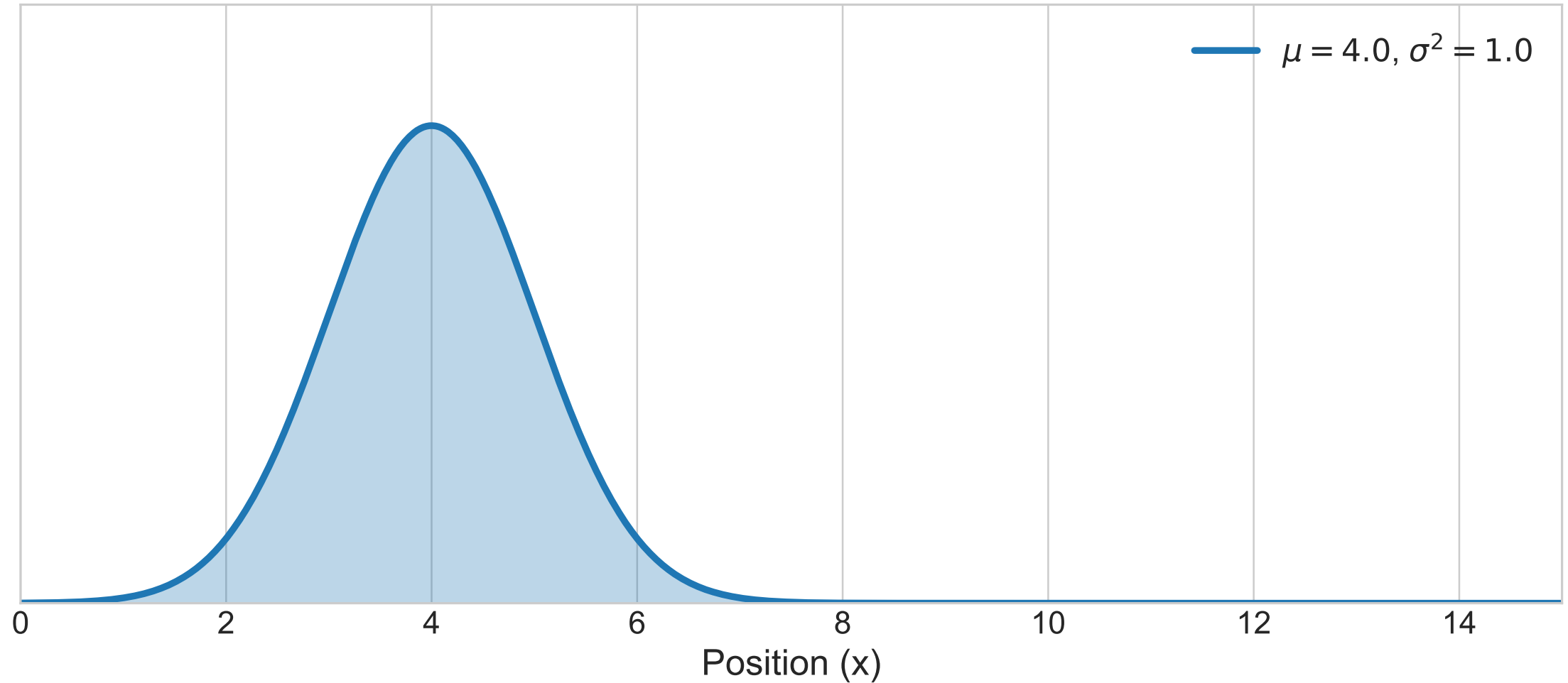
If we close our eyes and take 5 steps, are we more or less certain of our position than before we moved?

Before we look at massive matrices, let's look at a robot on a 1-Dimensional train track.

Its state is just one number: its position on the track, μ . Its uncertainty is also just one number: the variance, σ^2 .

Together, they form a 1D Gaussian (Bell Curve) representing our "belief".

Initial Belief



The robot commands its motors to move forward by distance u .

Our new predicted mean simply shifts:

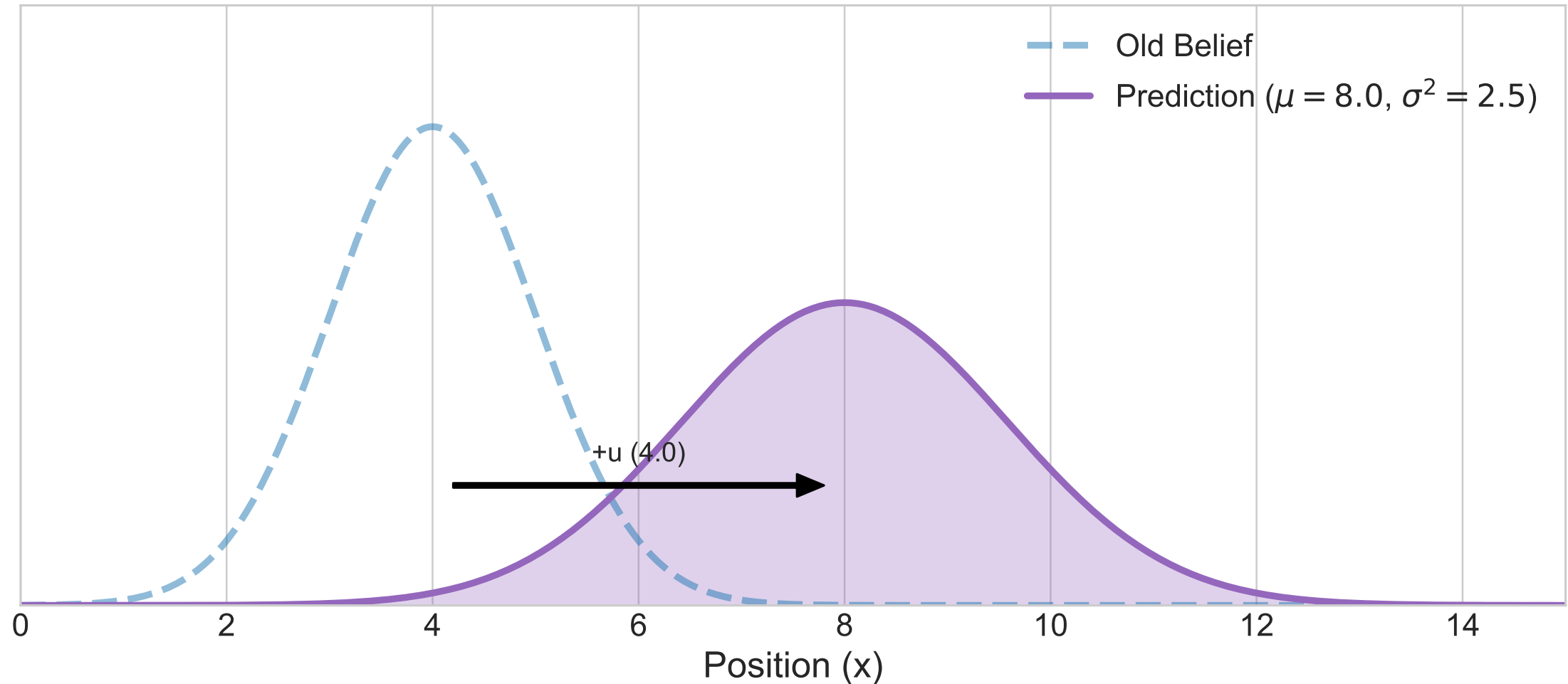
$$\mu_{pred} = \mu_{old} + u$$

But moving adds process noise (σ_{move}^2).

Our uncertainty grows.

$$\sigma_{pred}^2 = \sigma_{old}^2 + \sigma_{move}^2$$

1D Prediction (Shift & Spread)



This prediction updates the covariance and the means as such because we are assuming **Gaussian** random variables.

Let X and Y be independent random variables that are normally distributed, then their sum is also normally distributed. i.e., if

$$\begin{aligned} X &\sim N(\mu_X, \sigma_X^2), & Y &\sim N(\mu_Y, \sigma_Y^2) \\ Z &= X + Y \end{aligned}$$

Then,

$$Z \sim N(\mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$$

For how to derive it you can look [here \(from Wikipedia\)](#)

Update

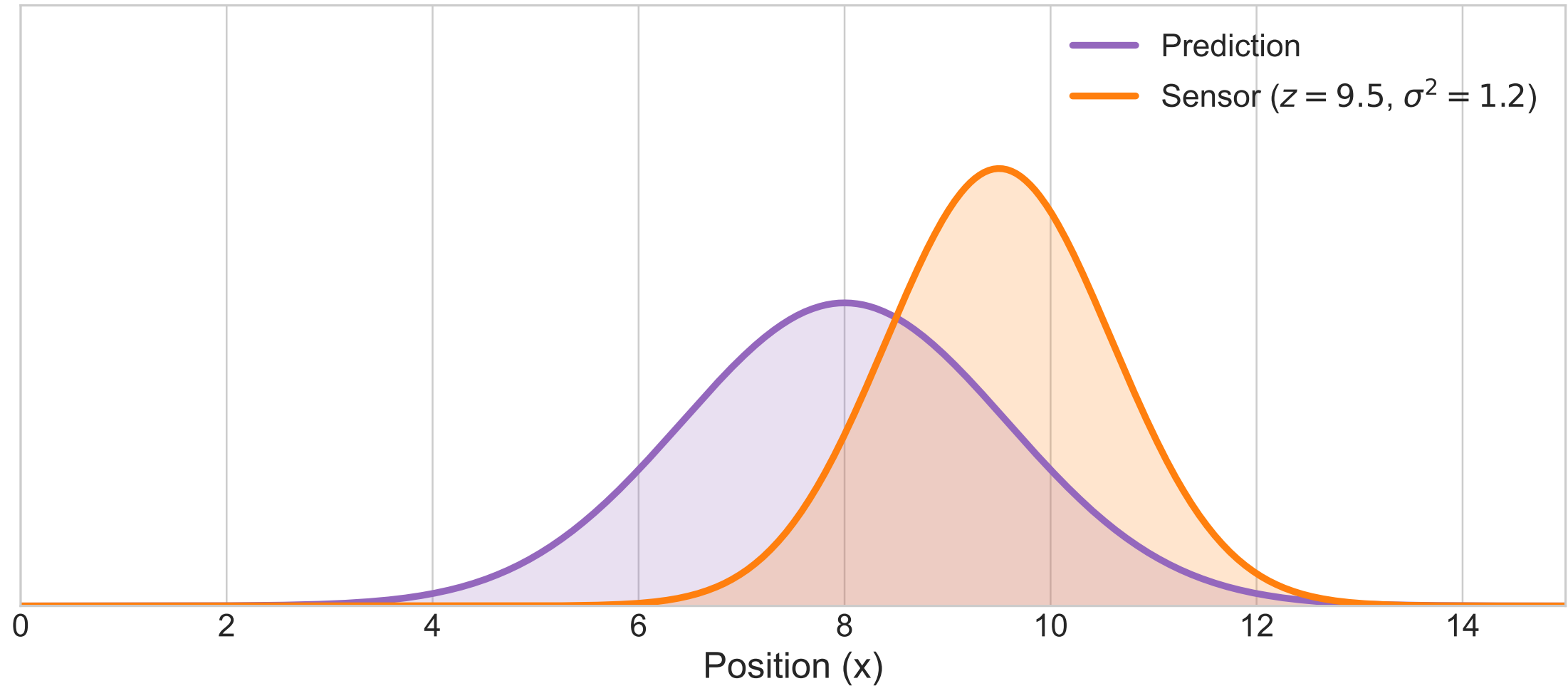
Now, our absolute sensor (like a 1D LiDAR) **takes a measurement**.

It says the robot is at position z , with a sensor noise variance of σ_{sensor}^2 .

We now have **two** Gaussian curves: our flattened **prediction** curve, and the new **sensor** curve.

How do we combine these two conflicting opinions into one single truth?

1D Update (Conflicting Opinions)



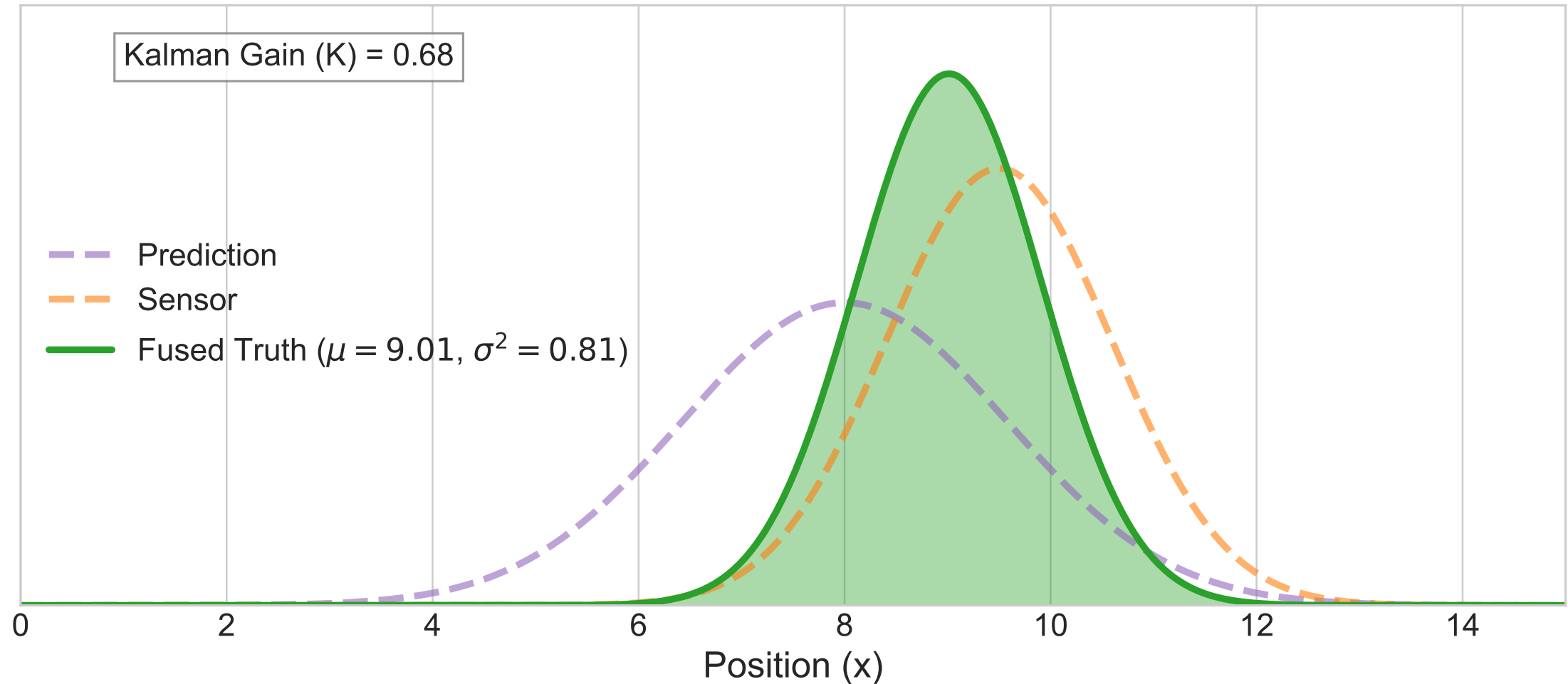
In probability, to fuse two independent estimates, we **multiply their distributions** together.

The **magical property** of Gaussians: Multiplying two Gaussians **always** produces a new **Gaussian**.

Because the new curve represents the overlapping area of agreement, it is always taller and narrower than both original curves.

Fusing data **reduces** uncertainty!

Fusing Data (Multiplying Gaussians)



The math behind that multiplication gives us the 1D **Kalman Gain** (K):

$$K = \frac{\sigma_{pred}^2}{\sigma_{pred}^2 + \sigma_{sensor}^2}$$

K is just a ratio! It's the ratio of our prediction uncertainty to the total system uncertainty. It always falls between 0 and 1.

We update our position using this ratio:

$$\mu_{new} = \mu_{pred} + K(z - \mu_{pred})$$

All it really says is “how much of the measurement should we incorporate”?

That's all the Kalman filter is!

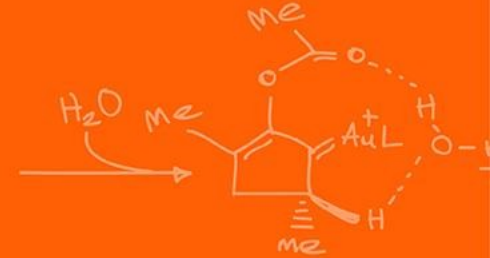
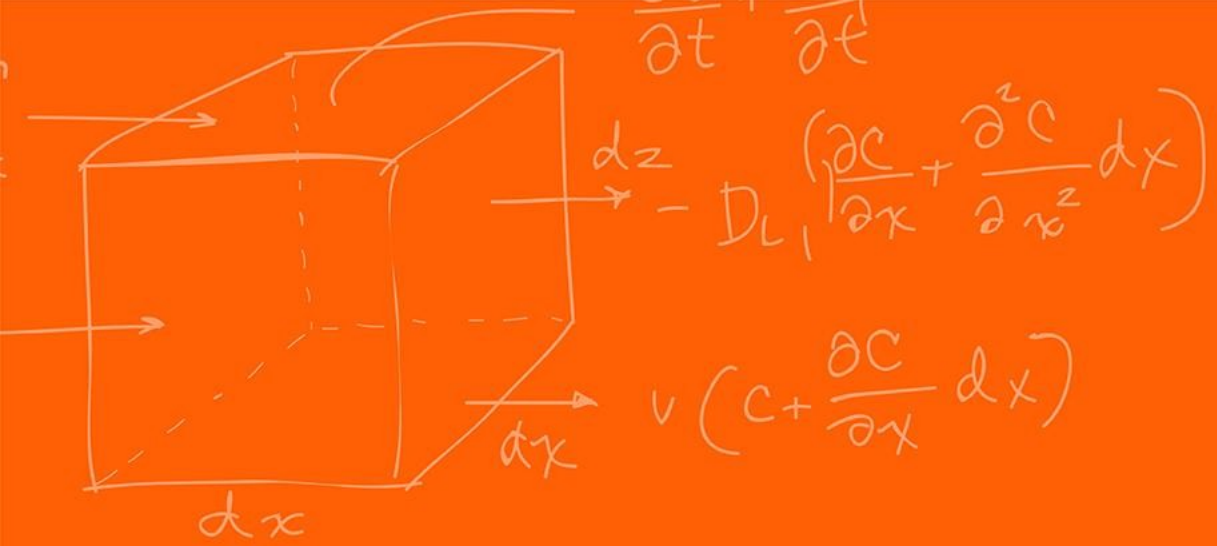
What is the current limitation of the formulation?

We are only looking at a problem with 1 state, can this be generalized to be N-dimensional?

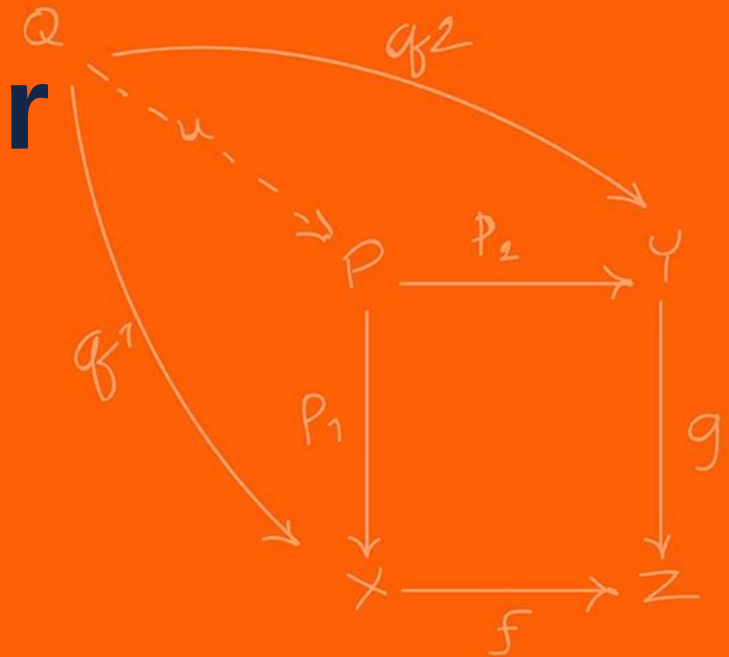
For our differentiable robot, we live in a 2D plane with an X , Y , and heading θ .

We can no longer use a single scalar number μ for position. We must use a **State Vector**: $x = [X, Y, \theta]^T$.

We can no longer use a single scalar variance σ^2 . We must use a **Covariance Matrix**, P .

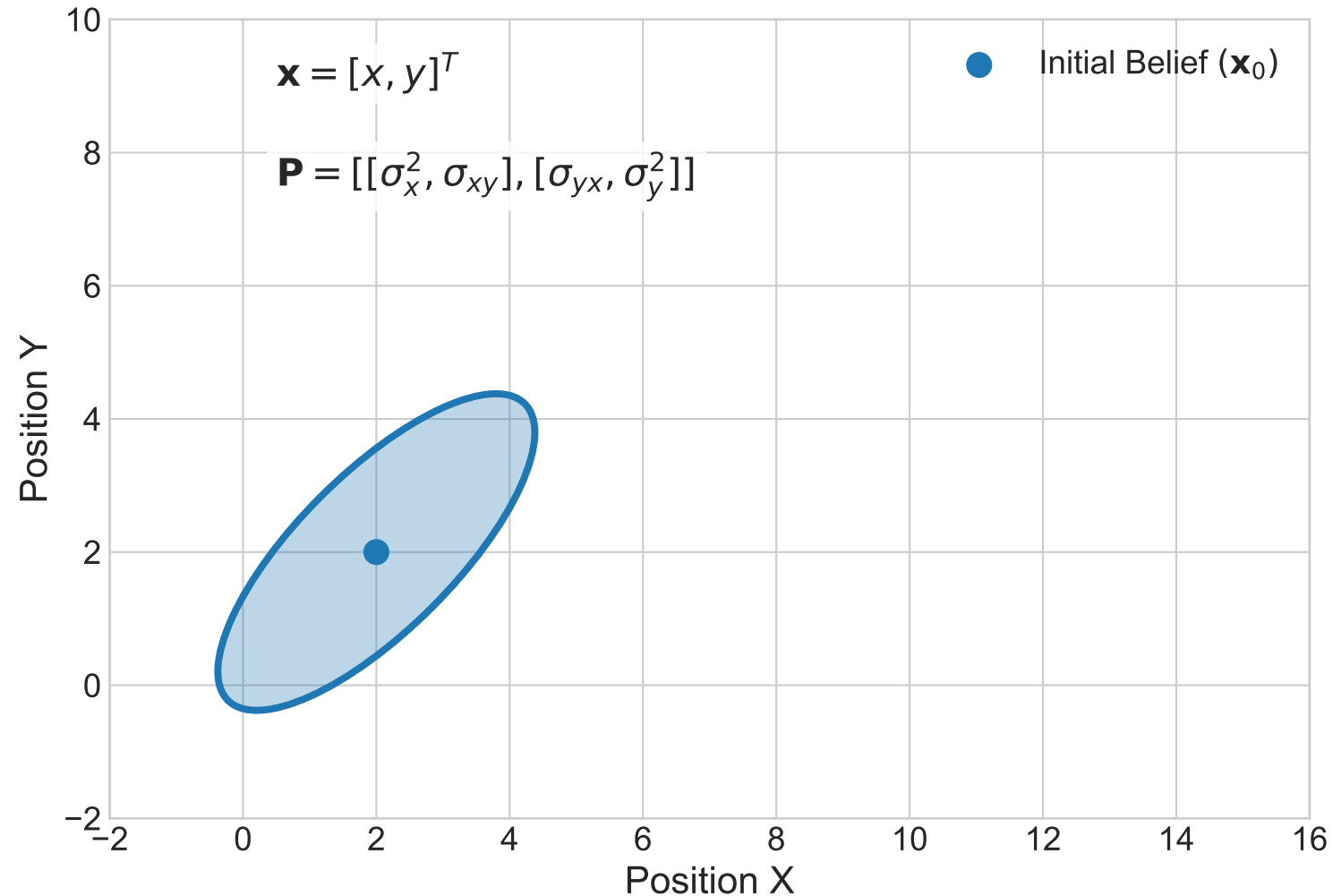


ND - Kalman Filter



Predict

MD Initial Belief (The State Vector)

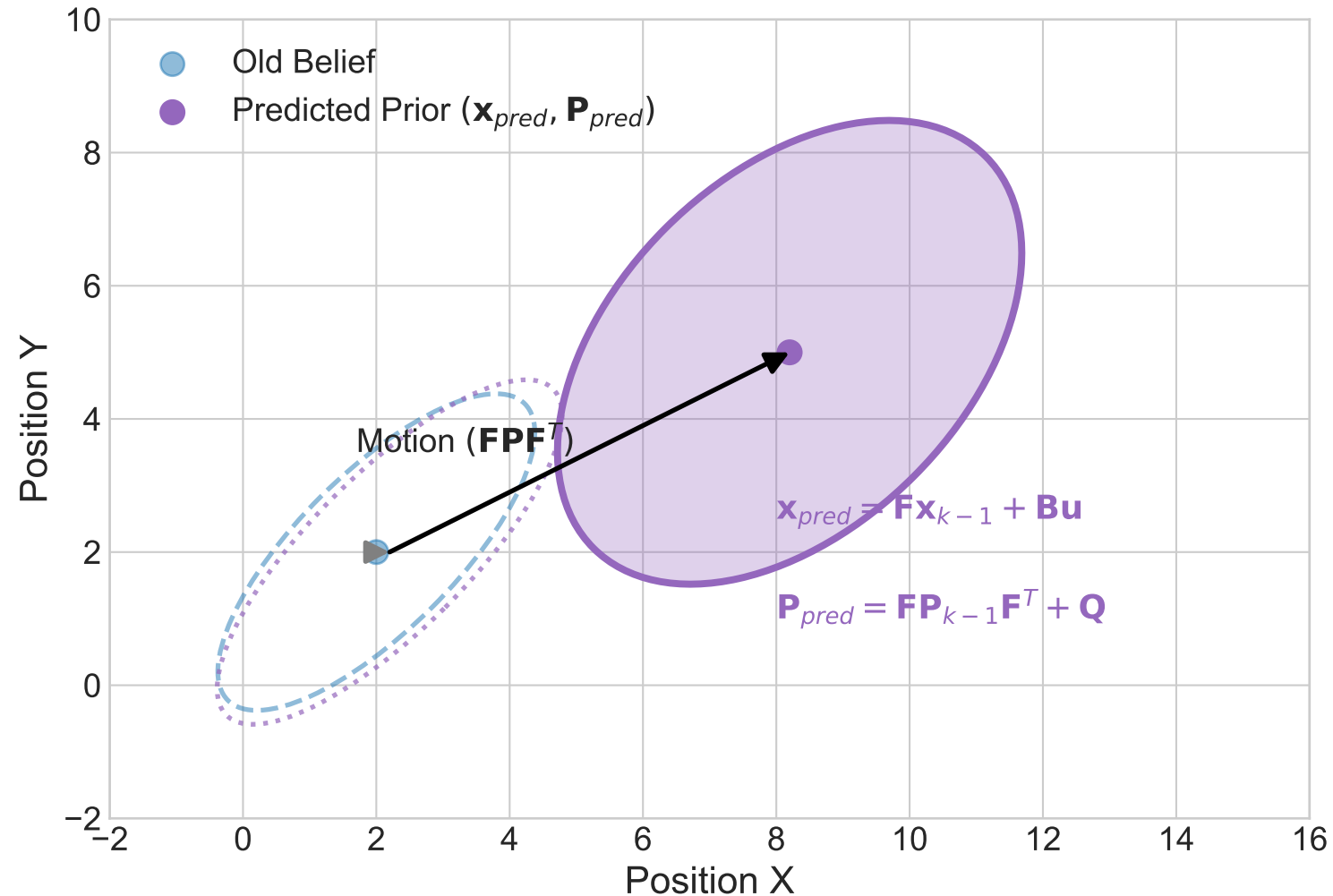


Let's translate our 1D prediction ($\mu_{pred} = \mu_{old} + u$) to N-Dimensions:

$$x_{k|k-1} = Fx_{k-1|k-1} + Bu_k$$

F is our state transition matrix (how physics updates the state).

MD Prediction (Linear Transformation)



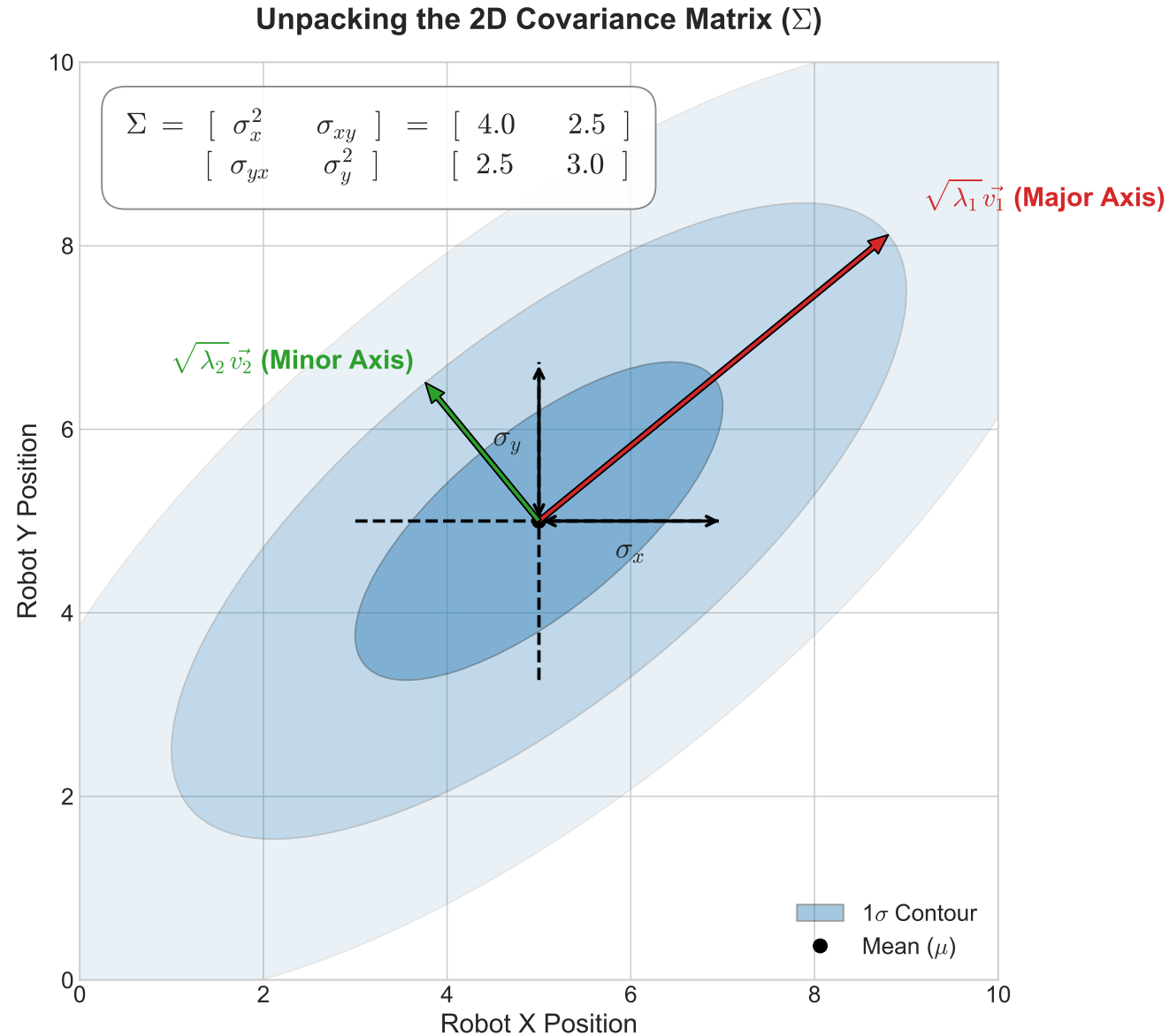
The **Covariance Matrix** P represents multi-dimensional uncertainty as an ellipse, not a 1D curve.

The diagonal elements are the individual variances (e.g., how unsure are we about X independent of everything else?).

The off-diagonal elements are the critical addition: **Correlations**.

If our robot only drives forward, and we are uncertain about our heading θ , will that create an uncertainty in Y as we travel?

Yes, errors in θ are highly correlated with errors in X and Y .



♥What is **Variance**, Really?

Before we calculate how uncertainty grows, we must define what it is.

Imagine throwing 100 darts at a board. The **Expected Value** ($E[X]$, or mean μ) is the center point of all your darts.

Variance measures **how spread out** those darts are from the center. It is the average of the squared distances from the mean.

We write this mathematically as:

$$Var(X) = E[(X - \mu)^2]$$

What happens to our uncertainty if we multiply our state by a constant a (like applying a gear ratio to a wheel)?

$$\text{Var}(aX) = E[(aX - E[aX])^2]$$

$$\text{Var}(aX) = E[(aX - a\mu)^2]$$

$$\text{Var}(aX) = E[a^2(X - \mu)^2]$$

Because a^2 is a constant, we pull it out:

$$\text{Var}(aX) = a^2 E[(X - \mu)^2]$$

Thus, $\text{Var}(aX) = a^2 \text{Var}(X)$. Scaling the state scales the uncertainty quadratically.

If a motor spins twice as fast, does the position uncertainty double?

No, it **quadruples!**

Real robots experience random bumps and slip. We represent this as adding a random noise variable W to our state.

Assuming the noise W is **completely independent** of our state X :

$$\text{Var}(aX + W) = \text{Var}(aX) + \text{Var}(W)$$

Substituting our previous proof:

$$\text{Var}(aX + W) = a^2 \text{Var}(X) + \text{Var}(W)$$

This simple 1D equation is the exact heart of the Kalman Filter's predict step!

Our 1D variance equation is $Var(X_{new}) = a^2 Var(X) + Var(W)$.

In a multidimensional robot, a becomes the State Transition Matrix F .

$Var(X)$ becomes the Covariance Matrix P .

$Var(W)$ becomes the Process Noise Matrix Q .

But how do we square a matrix to get the equivalent of a^2 ? We can't just write F^2 !

In linear algebra, the variance of a matrix transformation is derived using the transpose:

$$\text{Cov}(FX) = E[(FX - E[FX])(FX - E[FX])^T] = E \left[F(X - \mu)(F(X - \mu))^T \right]$$

By the rules of matrix transposition $(AB)^T = B^T A^T$:

$$= E[F(X - \mu)(X - \mu)^T F^T]$$

We pull the constant F matrices out of the Expectation:

$$\begin{aligned} &= FE[(X - \mu)(X - \mu)^T]F^T \\ &= FPF^T \end{aligned}$$

Adding our noise matrix Q , we get our final prediction: $P_{new} = FPF^T + Q$.

Update

♥ We've successfully predicted our new position, but our uncertainty (P) has grown.

Now, our sensor (e.g., LiDAR) provides a new measurement (z).

We have two imperfect guesses for our position: our prediction and our sensor. How do we fuse them? Think of a **Weighted Average**. If you and an expert both guess the weight of a robot, you don't just average the two numbers.

You give more weight to the expert.

$$Estimate = W_{model} \cdot Prediction + W_{sensor} \cdot Measurement$$

Again, for 1D the equation is:

$$\mu_{new} = \mu_{pred} + K(z - \mu_{pred})$$

Where we have:

$$K = \frac{\sigma_{pred}^2}{\sigma_{pred}^2 + \sigma_{sensor}^2}$$

If our sensor is perfect ($\sigma_{sensor}^2 = 0$), what does K become, and what happens to μ_{new} ?

$K = 1$, and our new estimate exactly equals the sensor measurement!

In 1D, state and measurement were both just position along a track. In 2D/3D, our state is $x = [X, Y, \theta]^T$. But our LiDAR might just measure the **straight-line distance** to a wall.

What is the problem?

We cannot subtract a state vector from a distance measurement! They speak different languages!

We introduce the **Observation Matrix** (H), which translates our internal state into the format of the sensor.

Expected Measurement = Hx_{pred}

We thus, start by modifying the 1D Kalman Gain into matrices.

$$K = \frac{\sigma_{pred}^2}{\sigma_{pred}^2 + \sigma_{sensor}^2}$$

σ_{pred}^2 becomes our covariance P .

σ_{sensor}^2 becomes our measurement noise R .

But we must map P into sensor space using H . Just like FPF^T for motion, we use HPH^T for sensors!

We thus, start by modifying the 1D Kalman Gain into matrices.

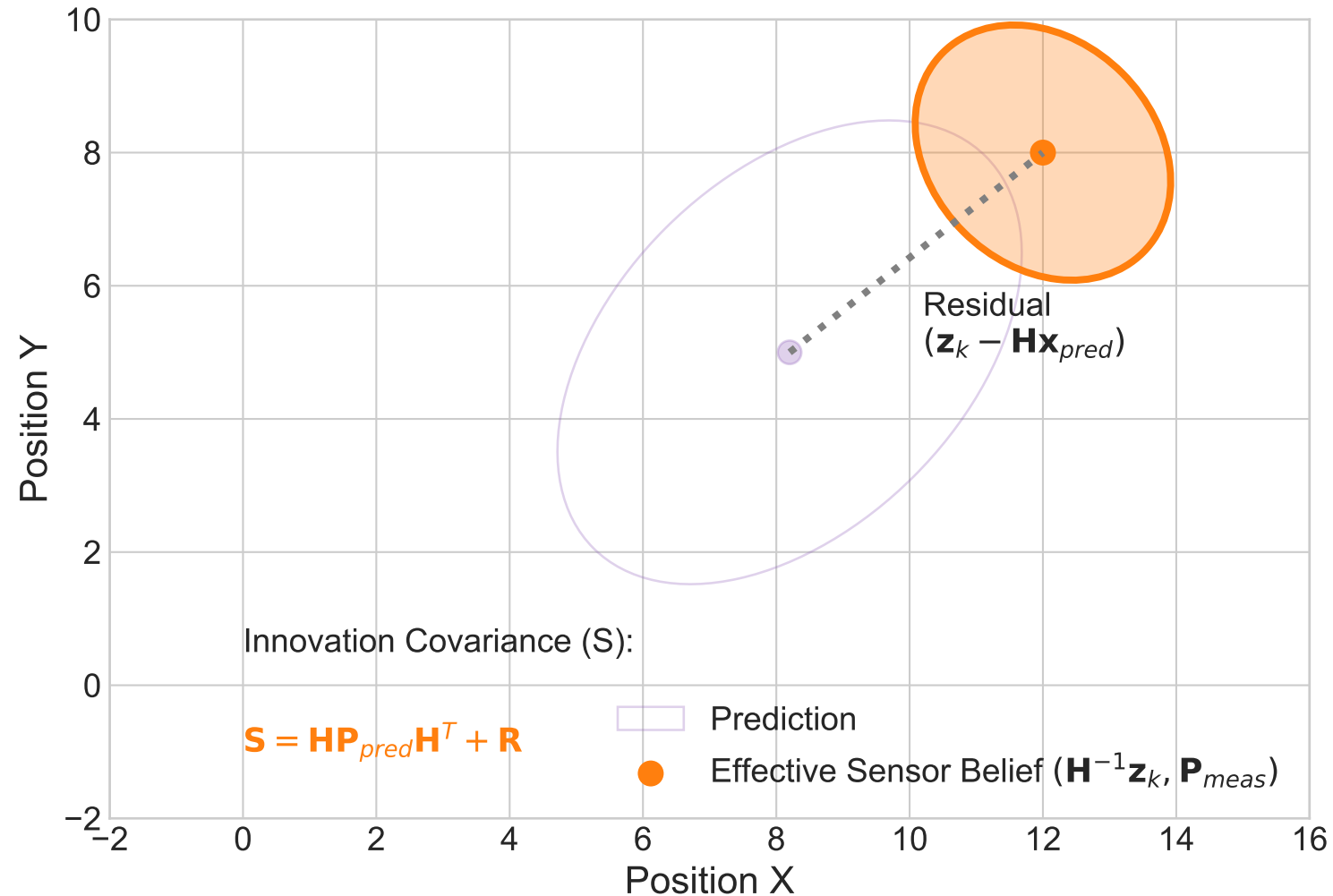
$$K = \frac{\sigma_{pred}^2}{\sigma_{pred}^2 + \sigma_{sensor}^2}$$

The numerator is the prediction covariance mapped to the sensor. The denominator is the total system noise.

Because we can't divide matrices, we multiply by the inverse:

$$K = PH^T (HPH^T + R)^{-1}$$

MD Update (Transforming the Measurement)



To derive the multi-dimensional math, we must define our ultimate goal: **Minimizing Uncertainty**.

Mathematically, total uncertainty is the **sum of the variances** of our state variables (the **diagonal elements of the covariance matrix P**).

The sum of the diagonal elements of a matrix is called the **Trace**, written $Tr(P)$.

Thus the objective is to find the Gain matrix, that forces the derive to achieve:

$$\frac{\partial Tr(P_{k|k})}{\partial K} = 0$$

Let's define the actual error of our updated state: $e = x_{true} - x_{k|k}$.

We substitute our update equation $x_{k|k} = x_{k|k-1} + K(z - Hx_{k|k-1})$ into the error definition.

Covariance is defined as the expected value of the error squared: $P_{k|k} = E[ee^T]$.

Expanding this massive expected value yields the universally true **Joseph Form of the covariance update**:

$$P_{k|k} = (I - KH)P_{k|k-1}(I - KH)^T + KRK^T$$

Why do we see both P (model noise) and R (sensor noise) in this equation?

Because our new estimate relies on both, so both sources contribute to the final error

• We have our covariance equation. Now, we use matrix calculus to minimize its Trace:

$$\frac{\partial \text{Tr}(P_{k|k})}{\partial K} = 0$$

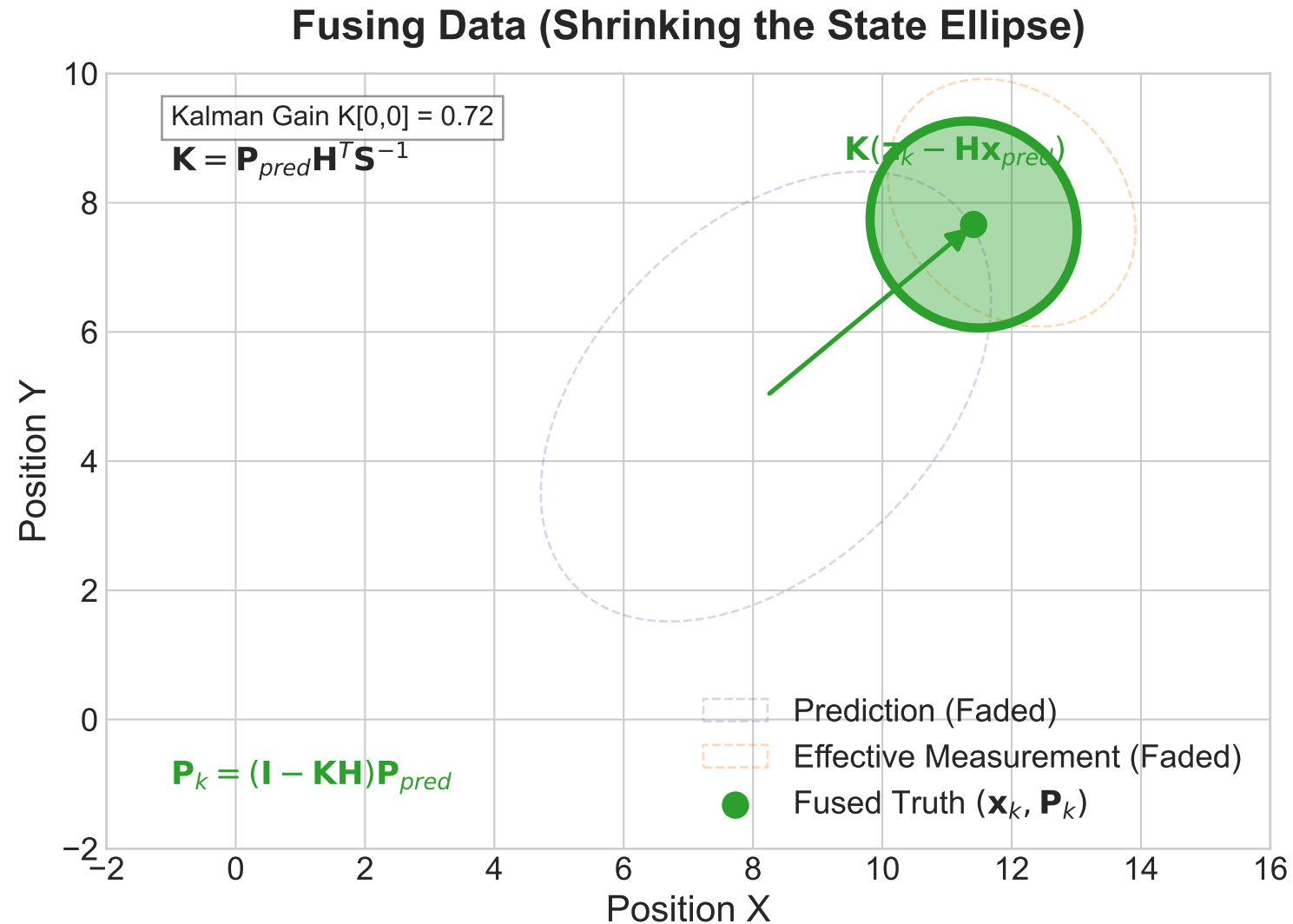
Taking the derivative of the Joseph Form with respect to K yields:

$$-2(I - KH)P_{k|k-1}H^T + 2KR = 0$$

Divide by 2, expand, and isolate the K terms:

$$P_{k|k-1}H^T = K(H P_{k|k-1}H^T + R) \text{ There it is!}$$

The $P_{k|k-1}H^T$ term emerges naturally from taking the derivative of the covariance. It represents the **cross-covariance between our state error and our measurement error**.



Now we apply our matrix K to update the multi-dimensional state.

The 1D equation $\mu_{new} = \mu_{pred} + K(z - \mu_{pred})$ becomes:

$$x_{k|k} = x_{k|k-1} + K(z_k - Hx_{k|k-1})$$

Finally, we update the uncertainty ellipse. We pull the information from the sensor out of our uncertainty:

$$P_{kk} = (I - KH)P_{kk-1}$$

(I is the Identity Matrix, the matrix equivalent of the number 1).
And we are done!

Now we have our optimal K . But calculating the massive Joseph Form

$$P_{k|k} = (I - KH)P_{k|k-1}(I - KH)^T + K RK^T$$

takes too much processing power on a microcontroller. Let's substitute our newly derived optimal K back into the Joseph Form.

Because $K(HPH^T + R) = PH^T$, massive sections of the Joseph equation perfectly cancel each other out.

The equation collapses into the short form:

$$P_{k|k} = (I - KH)P_{k|k-1}$$

But what does $(I - KH)$ mean physically?

Think of the Identity Matrix (I) as the number 1, representing 100% of our previous uncertainty ($P_{k|k-1}$).

H maps our state to the sensor.

K is how much we trust the sensor.

Therefore, KH is the **percentage of uncertainty the sensor measurement removes**.

If the sensor is completely useless (pure noise, $K=0$), what does the equation become?

$$P_{new} = (I - 0)P_{old} = P_{old}. \text{ Our uncertainty doesn't shrink at all!}$$

Prediction:

State Prediction (Where do we think we are?):

$$x_{k|k-1} = Fx_{k-1|k-1} + Bu_k$$

Covariance Prediction (How uncertain are we getting?):

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

Update:

The Kalman Gain (Who do we trust more?):

$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

State Update (Correcting our position):

$$x_{k|k} = x_{k|k-1} + K(z_k - Hx_{k|k-1})$$

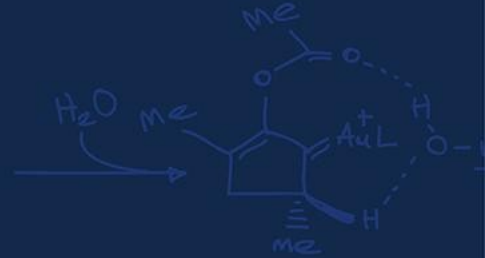
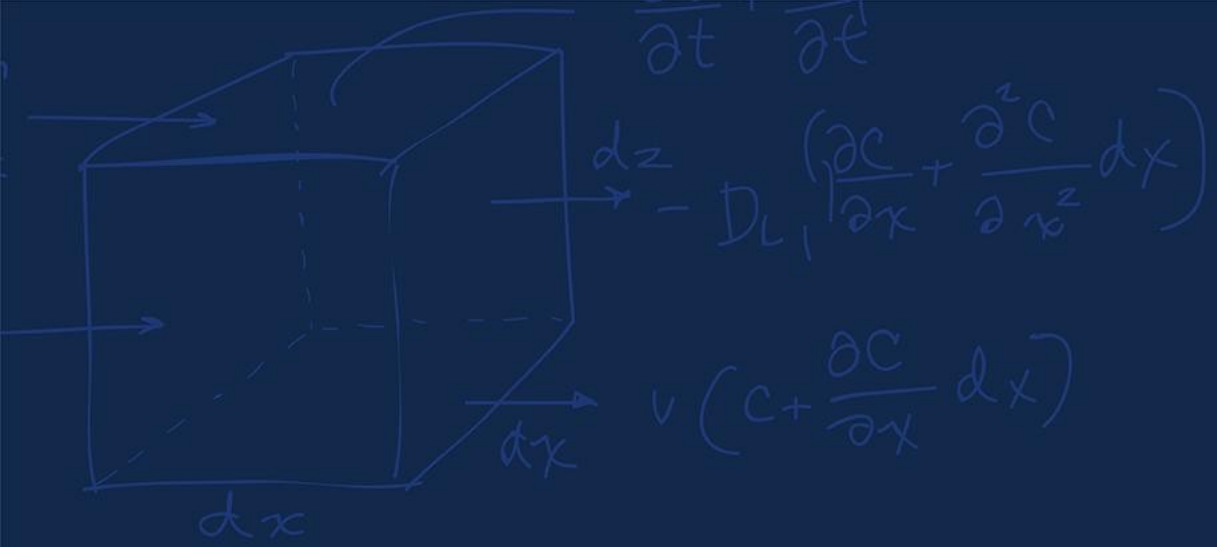
Covariance Update (Shrinking our uncertainty):

$$P_{k|k} = (I - KH)P_{k|k-1}$$

Sensor Fusion – Predict Step

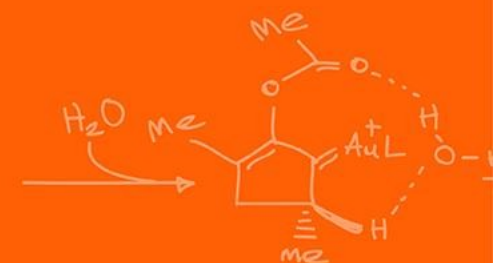
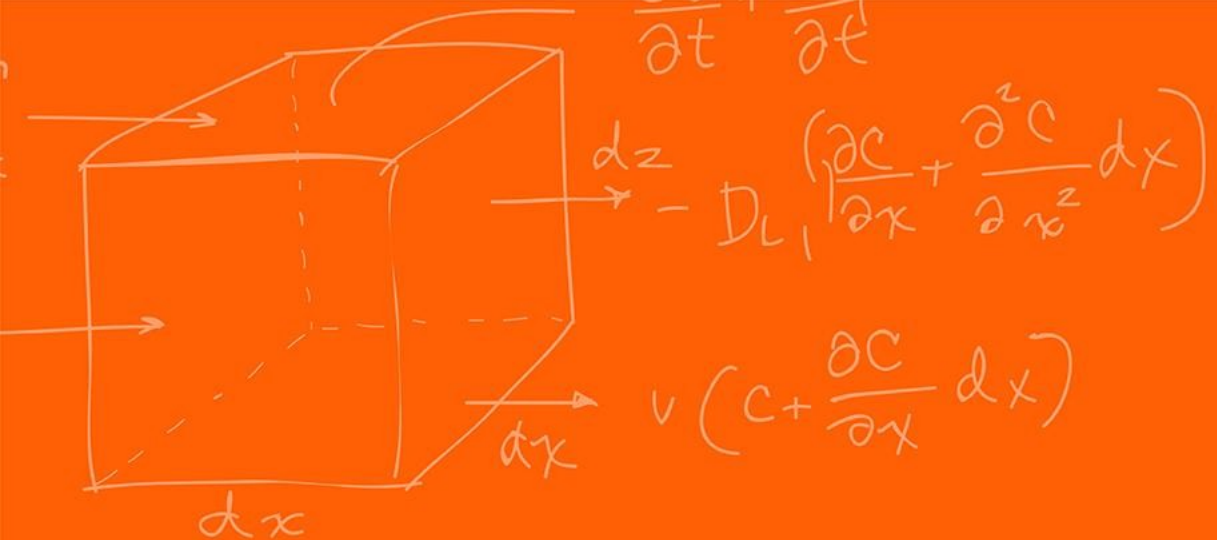


Concept	1D (Scalar)	N-dimensional (Matrix)	Description
State	State x The robot's current estimated condition (Position, Velocity, etc.)	State x The robot's current estimated condition (Position, Velocity, etc.)	The robot's current estimated condition (Position, Velocity, etc.)
Uncertainty	Uncertainty σ^2 How unsure we are about the state (Variance / Covariance Matrix)	Uncertainty P How unsure we are about the state (Variance / Covariance Matrix)	How unsure we are about the state (Variance / Covariance Matrix)
Physics Model	Physics Model F How the state naturally evolves over time (State Transition / Jacobian)	Physics Model F How the state naturally evolves over time (State Transition / Jacobian)	How the state naturally evolves over time (State Transition / Jacobian)
Process Noise	Process Noise Q Uncertainty added simply by moving / imperfections in the physics model	Process Noise Q Uncertainty added simply by moving / imperfections in the physics model	Uncertainty added simply by moving / imperfections in the physics model
Sensor Model	Sensor Model H Translates internal state space into the format the sensor outputs	Sensor Model H Translates internal state space into the format the sensor outputs	Translates internal state space into the format the sensor outputs
Sensor Noise	Sensor Noise R The inherent electrical/mechanical noise of the specific sensor	Sensor Noise R The inherent electrical/mechanical noise of the specific sensor	The inherent electrical/mechanical noise of the specific sensor
Measurement	Measurement z The raw data point provided by the sensor (LiDAR, Encoders, etc.)	Measurement z The raw data point provided by the sensor (LiDAR, Encoders, etc.)	The raw data point provided by the sensor (LiDAR, Encoders, etc.)
Kalman Gain	Kalman Gain $K = \frac{\sigma^2 F}{\sigma^2 F + R}$ The dynamic weight that decides whether to trust the model or the sensor	Kalman Gain $K = P F^T (P F^T + R)^{-1}$ The dynamic weight that decides whether to trust the model or the sensor	The dynamic weight that decides whether to trust the model or the sensor



Questions?





The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

